# Resumen de funciones usadas en la Práctica 1 (libpcap + Python)

Ordenado tal como aparecen en el flujo del programa, de arriba a abajo.

---

## 1. Funciones de `argparse` (leer argumentos de línea de comandos)

### `argparse.ArgumentParser(description=..., formatter_class=...)`

- **Cuándo se usa:** al principio del `main`, para crear el "lector" de argumentos.
- **Qué hace:** crea un objeto `parser` que sabe interpretar lo que el usuario escribe en la terminal (`--itf`, `--file`...).
- **En nuestro código:**

```
        parser = argparse.ArgumentParser(description='...',
formatter_class=RawTextHelpFormatter)
```

### `parser.add_argument('--nombre', dest=..., default=..., type=..., action=..., help=...)`

- **Cuándo se usa:** una vez por cada opción que queremos aceptar (`--file`, `--itf`, `--nbytes`, `--npkts`, `--debug`).
- **Qué hace:** define una opción concreta: cómo se llama, dónde se guarda su valor (`dest`), qué valor tiene por defecto, si se convierte a número (`type=int`), y si es un simple interruptor (`action='store_true'`).
- **En nuestro código:**

```
        parser.add_argument('--itf', dest='interface', default=False,
help='Interfaz a abrir')
        parser.add_argument('--npkts', dest='npkts', type=int,
default=-1, help='...')
        parser.add_argument('--debug', dest='debug', default=False,
action='store_true', help='...')
```

### `parser.parse_args()`

- **Cuándo se usa:** justo después de definir todas las opciones.
- **Qué hace:** lee de verdad lo que escribió el usuario y devuelve un objeto (`args`) con un campo por cada `dest` definido.
- **En nuestro código:**

```
        args = parser.parse_args()
        # luego: args.interface, args.tracefile, args.nbytes, args.npkts,
args.debug
```

### `parser.print_help()`

- **Cuándo se usa:** dentro del `if` que detecta que no se especificó ni `--file` ni `--itf`.

- **Qué hace:** imprime automáticamente el mensaje de ayuda (usage + lista de opciones), generado a partir de todos los `add_argument` anteriores.
- **En nuestro código:**

```
if args.tracefile is False and args.interface is False:
    logging.error('No se ha especificado interfaz ni fichero')
    parser.print_help()
    sys.exit(-1)
```

---

## 2. Funciones de `logging` (mensajes con nivel de importancia)

### `logging.basicConfig(level=..., format=...)`

- **Cuándo se usa:** justo después de leer `args`, dentro del `if args.debug: / else:`.
- **Qué hace:** configura el sistema de mensajes: qué nivel mínimo se muestra (DEBUG muestra todo; INFO oculta los DEBUG) y con qué formato de texto.
- **En nuestro código:**

```
        logging.basicConfig(level=logging.INFO, format='[%(asctime)s %
(levelname)s]\t%(message)s')
```

### `logging.debug(...)` / `logging.info(...)` / `logging.error(...)`

- **Cuándo se usa:** en varios puntos — al procesar cada paquete, al terminar, o si algo falla al abrir la interfaz/fichero.
- **Qué hace:** imprime un mensaje con ese nivel concreto de importancia. Solo se ve en pantalla si el nivel configurado en `basicConfig` lo permite.
- **En nuestro código:**

```
        logging.error('Error al abrir la interfaz {}:
{}'.format(args.interface, errbuf))
        logging.info('{} paquetes procesados'.format(num_paquete))
```

---

## 3. Funciones de `rc1_pcap.py` (el wrapper sobre la librería C)

### `pcap_open_live(device, snaplen, promisc, to_ms, errbuf)`

- **Cuándo se usa:** dentro del `if args.interface is not False:`, para modo `--itf`.
- **Qué hace:** abre una interfaz de red para captura en vivo (necesita `sudo`). Devuelve un `handle`, o `None` si falla.
- **En nuestro código:**

```
        handle = pcap_open_live(args.interface, ETH_FRAME_MAX, PROMISC,
TO_MS, errbuf)
```

### `pcap_open_offline(fname, errbuf)`

- **Cuándo se usa:** en el `else` del bloque anterior, para modo `--file`.

- **Qué hace:** abre una traza `.pcap` ya guardada para leerla. Devuelve un `handle`, o `None` si falla.
- **En nuestro código:**

```
handle = pcap_open_offline(args.tracefile, errbuf)
```

**`pcap_open_dead(linktype, snaplen)`**

- **Cuándo se usa:** solo en modo `--itf`, antes de crear los ficheros de salida.
- **Qué hace:** crea una "plantilla"/descriptor que define el TIPO de datos que se van a guardar (Ethernet = `DLT_EN10MB`) y el tamaño máximo por paquete. No crea ningún fichero todavía.
- **En nuestro código:**

```
descr_dump = pcap_open_dead(DLT_EN10MB, ETH_FRAME_MAX)
```

**`pcap_dump_open(descr, fname)`**

- **Cuándo se usa:** justo después de `pcap_open_dead`, dos veces (una por cada fichero de salida).
- **Qué hace:** abre de verdad el fichero de salida en disco, usando la plantilla creada antes. Devuelve un "dumper".
- **En nuestro código:**

```
pdumper_noip = pcap_dump_open(descr_dump, fname_noip)
pdumper = pcap_dump_open(descr_dump, fname_normal)
```

**`pcap_loop(handle, cnt, callback, user)`**

- **Cuándo se usa:** una única vez, es EL núcleo del programa, después de tener `handle` listo.
- **Qué hace:** empieza a leer paquetes de `handle`, llamando a `callback` (nuestra función `procesa_paquete`) por cada uno. `cnt` limita cuántos procesar (`-1` = ilimitado). Se queda "bloqueado" aquí hasta acabar los paquetes, llegar al límite, o recibir un `pcap_breakloop`.
- **En nuestro código:**

```
ret = pcap_loop(handle, args.npkts, procesa_paquete, None)
```

**`pcap_dump(dumper, header, data)`**

- **Cuándo se usa:** dentro de `procesa_paquete`, por cada paquete que cumple (o no) la condición de bytes 12/13.
- **Qué hace:** guarda literalmente un paquete (con su cabecera pcap y sus bytes) en el fichero de salida asociado al `dumper` indicado.
- **En nuestro código:**

```
if byte12 == 0x08 and byte13 == 0x06:
    pcap_dump(pdumper_noip, header, data)
else:
    pcap_dump(pdumper, header, data)
```

**`pcap_breakloop(handle)`**

- **Cuándo se usa:** dentro de `signal_handler`, cuando el usuario pulsa Ctrl+C.

- **Qué hace:** le pide a `pcap_loop` que pare de forma ordenada, en vez de que el programa muera en seco.
- **En nuestro código:**

```
def signal_handler(nsignal, frame):
    if handle:
        pcap_breakloop(handle)
```

**`pcap_close(handle)`**

- **Cuándo se usa:** al final del programa, después de que `pcap_loop` termine.
- **Qué hace:** cierra el descriptor de LECTURA (la interfaz o el fichero de origen que abrimos con `pcap_open_live`/`pcap_open_offline`).
- **En nuestro código:**

```
if handle:
    pcap_close(handle)
```

**`pcap_dump_close(dumper)`**

- **Cuándo se usa:** al final del programa, una vez por cada dumper que se haya creado.
- **Qué hace:** cierra el fichero de ESCRITURA, asegurando que todo lo guardado con `pcap_dump` se vuelca bien a disco.
- **En nuestro código:**

```
if pdumper:
    pcap_dump_close(pdumper)
if pdumper_noip:
    pcap_dump_close(pdumper_noip)
```

**Truco para no confundirlas:** `pcap_close` = cerrar por donde ENTRA el tráfico (lectura). `pcap_dump_close` = cerrar por donde SALE el tráfico guardado (escritura).

---

# 4. Funciones/módulos Python de propósito general

**`signal.signal(signal.SIGINT, signal_handler)`**

- **Cuándo se usa:** justo después de comprobar que hay `--file` o `--itf`.
- **Qué hace:** "conecta" la señal de Ctrl+C (`SIGINT`) con nuestra función `signal_handler`, para poder reaccionar a ella en vez de que el sistema operativo mate el programa directamente.

**`time.time()`**

- **Cuándo se usa:** una vez, al construir los nombres de los ficheros de salida (solo en modo `--itf`).
- **Qué hace:** devuelve el tiempo actual en segundos Unix (desde 1970), con decimales. Lo envolvemos en `int(...)` para quitar los decimales.
- **En nuestro código:**

```
fecha = int(time.time())
```

**`binascii.hexlify(bytes)`**

- **Cuándo se usa:** dentro de la impresión de bytes en hexadecimal (si usaste esta variante en vez de formatear byte a byte).
- **Qué hace:** convierte una secuencia de bytes a su representación en texto hexadecimal.

**`sys.exit(código)`**

- **Cuándo se usa:** cada vez que detectamos un error irrecuperable (sin argumentos, fallo al abrir interfaz/fichero).
- **Qué hace:** termina el programa inmediatamente, devolviendo ese código de salida (0 = éxito, distinto de 0 = error).

---

# 5. La palabra clave `global`

- **Cuándo se usa:** al principio de `procesa_paquete` y de `signal_handler` (para leer/modificar variables definidas fuera de la función: `num_paquete`, `pdumper`, `pdumper_noip`, `handle`, `args`, `primer_ts`, `ultimo_ts`).
- **Qué hace:** le dice a Python "esta variable no es nueva, es la misma que existe fuera de esta función, y quiero poder modificarla aquí dentro". Sin `global`, cualquier asignación crearía una copia local nueva que no afectaría a la variable de fuera.
- **Regla práctica:** solo hace falta si vas a REASIGNAR (`variable = algo`) esa variable dentro de la función. Si solo la LEES (sin asignarla), no hace falta declararla `global`.

---

# 6. Orden completo del flujo del programa (para verlo todo junto)

```
1. Se define el parser y se leen los argumentos (args)
2. Se configura logging según --debug
3. Si no hay --file ni --itf → ayuda y salir
4. Se conecta Ctrl+C con signal_handler
5. Según --itf o --file:
     a) --itf: pcap_open_live + pcap_open_dead + 2×pcap_dump_open
     b) --file: pcap_open_offline
6. pcap_loop(handle, args.npkts, procesa_paquete, None)
   └─ por cada paquete → procesa_paquete():
        - actualiza primer_ts / ultimo_ts
        - imprime los N primeros bytes en hex
        - si --itf: guarda el paquete en el dumper correspondiente
(pcap_dump)
7. Al terminar pcap_loop:
     - imprime nº de paquetes procesados
     - imprime diferencia de tiempo (o 0 si <2 paquetes)
     - cierra dumpers (pcap_dump_close) y handle (pcap_close)
```